



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

پشتیبانی از شبکه در سیستم عامل XV6

اعضای گروه

پارمیس حمصیان اتفاق

سبحان آقاسی زاده

پارسا حاجی قاسمی

علی امینی

نام درس

سیستم‌های عامل

نیم‌سال اول ۱۴۰۴-۱۴۰۵

نام استاد درس

دکتر اسدی

چکیده: این گزارش روند پیاده‌سازی و ارزیابی زیرسیستم شبکه در سطح هسته سیستم عامل xv6 را به تفصیل شرح می‌دهد. اتصال سیستم عامل به شبکه، درایور سخت‌افزاری E1000، مسیر کامل انتقال و دریافت داده‌ها با بهره‌گیری از دسترسی مستقیم به حافظه، همگام‌سازی سوکت‌ها و در نهایت پشتیبانی از پیام‌های کنترلی خطا در این پروژه پیاده‌سازی شده‌اند.

واژه‌های کلیدی: سیستم عامل xv6، شبکه اترنت، درایور E1000، پروتکل ICMP

۱ مقدمه و اهداف پروژه

نخستین بخش این پژوهش به برقراری ارتباط شبکه‌ای سیستم عامل از طریق یک دستگاه شبکه مجازی به نام E1000 می‌پردازد. این کارت شبکه به یک شبکه محلی اترنت که توسط نرم‌افزار شبیه‌ساز QEMU فراهم شده است متصل می‌گردد. در این معماری، سیستم عامل xv6 به عنوان سیستم مهمان دارای آدرس شبکه ۱۵۰.۲۰۰.۱۰ تنظیم شده است و بسته‌های داده خود را به آدرس ۲۰۲.۰۰.۱۰ که نماینده سیستم میزبان است ارسال می‌کند. هدف نهایی درک مفهومی و پیاده‌سازی عملی زیرسیستم شبکه در سیستم عامل‌های مدرن است.

۱-۱ محدودیت‌های شبکه و مدیریت حافظه

در طراحی این زیرسیستم مقادیر خاصی برای اندازه بسته‌ها در نظر گرفته شده است تا تطابق با استانداردها حفظ شود. کابل‌های استاندارد اترنت دارای حداکثر واحد انتقال (MTU) برابر با ۱۵۰۰ بایت هستند که به معنای بزرگ‌ترین بسته مجاز روی سیم است. با کسر حداقل ۲۰ بایت برای سرآیند پروتکل IP و ۸ بایت برای سرآیند پروتکل UDP، فضای خالص باقیمانده برای بار داده ۱۴۷۲ بایت خواهد بود. برای اطمینان بیشتر و جلوگیری از قطعه‌قطعه شدن بسته‌ها یا سرریز بافر، این مقدار روی عدد امن ۱۴۰۰ بایت تنظیم شده است. این ثابت تعیین می‌کند که بافرهای تخصیص یافته در صف سوکت چه ابعادی داشته باشند.

از سوی دیگر، به دلیل تفاوت در معماری ذخیره‌سازی بایت‌ها، سیستم عامل نیاز به اصلاح ترتیب آن‌ها دارد. پردازنده RISC-V از معماری بایت کم‌ارزش‌تر در ابتدا استفاده می‌کند، در حالی که شبکه از استاندارد بایت باارزش‌تر در ابتدا بهره می‌برد. به همین دلیل، مقادیر چندبایتی در سرآیندها با استفاده از توابع تبدیلی مانند ntohs برای مقادیر دو بایتی و ntohl برای مقادیر چهار بایتی اصلاح می‌شوند.

۲ معماری لایه‌ها و مسیرهای انتقال

۱-۲ مسیر ارسال داده به شبکه

فرایند ارسال بسته‌ها از فضای پردازش کاربر آغاز می‌شود. برنامه کاربر فراخوانی سیستمی send را با پارامترهای پورت مبدا و مقصد، آدرس مقصد و بافر داده صدا می‌زند. تابع sys_send در سطح هسته، یک صفحه جدید از حافظه اصلی را با استفاده از دستور kalloc تخصیص می‌دهد. این تابع سرآیندهای UDP، IP و اترنت را به ترتیب به داده خام افزوده و کپسوله‌سازی را انجام می‌دهد، سپس بافر آماده شده را به تابع درایور e1000_transmit می‌سپارد.

در سطح درایور، تابع فرستنده ابتدا قفل e1000_lock را دریافت می‌کند تا از تداخلات ناشی از پردازش‌های همزمان جلوگیری شود. سیستم با خواندن ثبات کنترلی E1000_TDT، جایگاه خالی بعدی را در حلقه ارسال توصیف‌گرها پیدا می‌کند. در این مرحله با بررسی بیت DD وضعیت بسته قبلی در آن جایگاه کنترل می‌شود و اگر سخت‌افزار کار ارسال آن را تمام کرده باشد، بافر قدیمی با استفاده از تابع kfree پاکسازی می‌شود که به این عمل پاکسازی تنبل می‌گویند.

سپس آدرس فیزیکی بسته جدید در توصیف‌گر سخت‌افزاری حلقه ثبت می‌گردد و پرچم‌های پایان بسته و درخواست گزارش وضعیت تنظیم می‌شوند. با به‌روزرسانی ثبات Tail، سخت‌افزار از وجود بسته جدید مطلع می‌شود. کارت E1000 از طریق مکانیزم دسترسی مستقیم به حافظه (DMA)، داده‌ها را بدون درگیر کردن پردازنده اصلی از حافظه می‌خواند و به صورت سیگنال‌های الکتریکی روی شبکه می‌فرستد. در پایان، سخت‌افزار وضعیت اتمام را در حافظه بازنویسی می‌کند تا فضای بافر برای چرخه‌های بعدی آزاد شود.

۲-۲ مسیر دریافت داده از شبکه

مسیر دریافت با رسیدن سیگنال‌های الکتریکی به کارت سخت‌افزاری E1000 آغاز می‌گردد. سخت‌افزار پس از یافتن یک بافر خالی در حلقه دریافت، داده‌ها را مستقیماً از طریق DMA در حافظه سیستم (RAM) نوشته و روی خط پردازنده یک سیگنال وقفه تولید می‌کند. پردازنده با متوقف کردن کار فعلی خود وارد توابع مدیریت وقفه یعنی usertrap و devintr می‌شود که وظیفه تشخیص منبع وقفه را دارند.

با تایید اینکه وقفه متعلق به شبکه است، روتین e1000_intr اجرا شده و تابع اصلی دریافت یعنی

e1000_recv فراخوانی می‌شود. این تابع پس از بررسی تاییدیه توصیف‌گر، بافر حاوی پیام را به لایه بالاتر یعنی تابع net_rx ارسال کرده و فوراً یک بافر خالی جدید برای جلوگیری از توقف دریافت جایگزین می‌کند. در لایه شبکه هسته، تابع net_rx نوع بسته را از روی سرآیند اترنت تحلیل کرده و در صورت مطابقت با پروتکل IP، آن را به روتر داخلی سیستم یعنی تابع ip_rx تحویل می‌دهد.

تابع ip_rx پس از عبور از سرآیندهای اولیه، نوع پروتکل را استخراج کرده و در صورت مشاهده بسته UDP، پورت مقصد آن را می‌خواند. سیستم با استفاده از تابع find_socket، سوکت متصل به این پورت را جستجو کرده و پس از یافتن آن، اشاره‌گر بافر را در انتهای صف داده‌های سوکت درج می‌کند. بلافاصله تابع wakeup فراخوانی می‌شود تا پروسه کاربری که در وضعیت خواب منتظر این بسته بوده است بیدار شود. در نهایت، تابع sys_recv در فضای کاربر متوجه بسته جدید در صف شده، آن را خارج کرده و داده‌های خالص را با استفاده از تابع copyout به حافظه برنامه کپی می‌کند. بافر اصلی هسته در انتها توسط تابع kfree پاکسازی می‌شود. بسته‌های اضافی در صورت پر بودن صف سوکت دور انداخته شده و سریعاً آزاد می‌گردند تا از مصرف بی‌رویه و تخریب حافظه جلوگیری شود. هر سوکت توانایی نگهداری حداکثر ۱۶ بسته در حال انتظار را دارد.

۳ همگام‌سازی و قفل‌ها

به منظور دستیابی به کارایی بالا و جلوگیری از تداخل برنامه‌ها، پشته شبکه از معماری قفل‌های تفکیک‌شده بهره می‌برد. یک قفل سراسری به نام sock_lock در سیستم وجود دارد که منحصراً از ساختار کلی آرایه سوکت‌ها و لیست اتصالات باز محافظت می‌کند. این قفل تنها در مواقع حساس ایجاد سوکت جدید از طریق فراخوانی bind یا از بین بردن سوکت از طریق فراخوانی unbind درگیر می‌شود تا از ثبت همزمان دو پردازش در یک درگاه جلوگیری کند.

پس از تخصیص سوکت، پردازش‌ها به قفل سراسری نیازی ندارند و از قفل‌های اختصاصی استفاده می‌کنند. برای هر سوکت یک قفل جداگانه در نظر گرفته شده است که صرفاً از متغیرهای مرتبط با صف همان سوکت شامل نشانگرهای ابتدا و انتها محافظت می‌کند. این قفل اختصاصی هر زمان که وقفه دریافت سخت‌افزاری قصد درج بسته جدیدی را داشته باشد و همزمان برنامه کاربر بخواهد بسته‌ای را از صف بخواند، کنترل را به دست می‌گیرد تا از خرابی حافظه ناشی از دسترسی متقاطع پیشگیری کند. تفکیک قفل‌ها این مزیت کلیدی را دارد که پردازش‌های مختلف می‌توانند به طور کاملاً موازی و بدون مسدود کردن یکدیگر روی سوکت‌های متفاوت کار کنند که این امر سرعت کلی سیستم را به شکل چشمگیری افزایش می‌دهد. علاوه بر موارد فوق، سیستم عامل دارای یک

قفل عمومی قدیمی به نام netlock است که در این پروژه مقداردهی می‌شود تا در صورت فراخوانی شدن توسط سایر بخش‌های جانبی هسته، سیستم دچار فروپاشی نشود.

۴ ارتقای شبکه و ICMP

پشته شبکه در پیاده‌سازی اولیه تنها از پروتکل‌های انتقال استاندارد پشتیبانی می‌کند، اما در بخش ارتقایافته، قابلیت درک پیام‌های خطای شبکه یا همان پروتکل ICMP به هسته اضافه شده است. این امکان موجب شفافیت بیشتر در مسیر ارتباطی می‌شود و به برنامه‌های کاربر اجازه می‌دهد تا از بسته بودن درگاه‌های مقصد مطلع گردند. جهت انتقال وضعیت خطا از لایه زیرین هسته به برنامه، یک فیلد عدد صحیح به نام err به ساختار اصلی سوکت‌ها افزوده شد.

```
struct sock {  
    int port;  
    int valid;  
    struct spinlock lock;  
    int err;  
};
```

مسیر دریافت شبکه با اعمال یک شرط جدید اصلاح گردید تا بسته‌هایی که مشخصه پروتکل آن‌ها برابر با عدد یک است، به جای هدایت به تابع IP، در اختیار تابع اختصاصی icmp_rx قرار گیرند. این تابع بسته‌های کنترلی را تحلیل کرده و پیام‌های نوع سه که بیانگر عدم دسترسی به مقصد هستند را فیلتر می‌کند. از آنجایی که پیام خطای ICMP حاوی بخشی از بسته اولیه ایجادکننده خطا است، تابع لایه‌های درونی را باز کرده و پورت مبدا را استخراج می‌کند. سپس با فراخوانی تابع wakeup، پردازشی که روی این درگاه معلق مانده است بیدار می‌شود. در کنار این تغییرات، منطق توابع مسدودشونده دریافت داده نیز ارتقا یافته است به نحوی که متغیر خطای سوکت را پیوسته پایش کرده و در صورت فعال شدن آن، به جای مسدود ماندن دائمی، مقدار بازگشتی منفی یک را به برنامه بازمی‌گردانند.

۵ آزمایش و ارزیابی

برای تایید کارایی و صحت کدهای پیاده‌سازی شده، مجموعه‌ای از اسکریپت‌های نوشته شده به زبان پایتون مورد استفاده قرار می‌گیرند. اسکریپت `nettest.py` ابزاری است که با استفاده از سوکت‌های استاندارد لایه کاربر در سیستم میزبان، شرایط مختلف ترافیکی را شبیه‌سازی می‌کند. این اسکریپت عملکردهای متنوعی مانند گوش دادن به یک بسته خروجی، ارسال یک بسته ورودی برای تحریک سخت‌افزار، تولید جریان‌های مداوم داده برای تست پایداری، تفکیک همزمان پورت‌های مختلف و آزمایش پینگ بازگشتی را به اجرا درمی‌آورد.

برای اتوماسیون فرایند ارزیابی، اسکریپت `test-xv6.py` طراحی شده است که وظیفه راه‌اندازی شبیه‌ساز QEMU بدون نیاز به تعامل دستی را بر عهده دارد. این برنامه خروجی‌های خط فرمان ماشین مجازی را ثبت کرده و از طریق عبارات منظم متنی، موفقیت یا شکست تست‌ها را تحلیل می‌کند و در شرایط خاص توانایی شبیه‌سازی قطعی و خطای سیستم را نیز داراست. هماهنگی کلی این اسکریپت‌ها بر عهده کتابخانه `gradelib.py` است که توابع اساسی کامپایل سیستم، اتصال به درگاه‌های اشکال‌زدایی GDB و ثبت ریز نمرات را فراهم می‌آورد. خروجی اجرای موفق فرایندهای ارزیابی در تصویر زیر قابل مشاهده است.

```
$ make qemu-gdb
(26.8s)
== Test    nettest: txone ==
    nettest: txone: OK
== Test    nettest: arp_rx ==
    nettest: arp_rx: OK
== Test    nettest: ip_rx ==
    nettest: ip_rx: OK
== Test    nettest: ping0 ==
    nettest: ping0: OK
== Test    nettest: ping1 ==
    nettest: ping1: OK
== Test    nettest: ping2 ==
    nettest: ping2: OK
== Test    nettest: ping3 ==
    nettest: ping3: OK
== Test    nettest: dns ==
    nettest: dns: OK
== Test    nettest: free ==
    nettest: free: OK
== Test time ==
time: FAIL
    Cannot read time.txt
Score: 170/171
make: *** [Makefile:370: grade] Error 1
```